The queue from chapter 4 shows a way of incorporating such waiting into the data structure itself using a condition variable inside the data structure, so let's look at that next.

6.2.2. A thread-safe queue using locks and condition variables

The thread-safe queue from chapter 4 is reproduced in listing 6.2. Much like the stack was modeled after `std::stack<>`, this queue is modeled after `std::queue<>`. Again, the interface differs from that of the standard container adaptor because of the constraints of writing a data structure that's safe for concurrent access from multiple threads.

Listing 6.2. The full class definition for a thread-safe queue using condition variables

```
template<typename T>
class threadsafe_queue
{
private:
    mutable std::mutex mut;
    std::queue<T> data_queue;
    std::condition_variable data_cond;
public:
    threadsafe_queue()
    {}

    void push(T new_value)
    {
        std::lock_guard<std::mutex> lk(mut);
        data_queue.push(std::move(data));
        data_cond.notify_one();          <-1
    }

    void wait_and_pop(T& value)    <-2
    {
        std::unique_lock<std::mutex> lk(mut);
        data_cond.wait(lk,[this]{return !data_queue.empty();});
        value=std::move(data_queue.front());
        data_queue.pop();
    }
```